

Workforce Frontend Handoff

Polished PDF master manual for Manus and frontend delivery

Bundle contents

Prepared for: **Workforce frontend design and implementation**

Document: **WORKFORCE_MANUS_FRONTEND_HANDOFF_COMBINED.md**

Use case: **Manus / frontend developer handoff**

This PDF set is designed as a frontend design and implementation handoff. The live OpenAPI remains the runtime contract authority wherever live behavior differs from planning notes.

Workforce Frontend Build — Master Prompt for Manus

You are building the first functional browser-visible frontend for the Workforce platform.

Your job is to create a production-minded frontend shell and connected operational pages that work against the current Workforce API, while preserving the architecture needed for the larger multi-tenant Workforce + Hospitable product.

Primary objective

Build a clean, modular, permission-aware web console that:

- authenticates against the current Workforce API,
- hydrates the session from the backend,
- supports business-context switching,
- renders current live data pages for rooms, tasks, assignments, and shifts,
- is structured so future Workforce and Hospitable modules can be added without rewriting the shell.

Product framing

Treat Workforce as the main platform and Hospitable / HKOps as a workspace inside it, not a separate standalone product.

The long-term platform supports:

- multi-business tenancy,
- location-scoped RBAC,
- worker self-service,
- business and location administration,
- agent/integration access,
- operational workspaces such as Hospitable.

Do not build a disconnected hospitality-only app. Build a shared Workforce console that can host multiple workspaces.

Source-of-truth rules

- 1 The live OpenAPI contract is the source of truth for currently available endpoints.
- 2 The planning documents define the future shape of the frontend and the future module/workspace structure.
- 3 If a future page is not yet supported by the live API, structure the route and UI shell for it, but do not fake backend capabilities unless clearly marked as placeholder or scaffold.
- 4 Do not hard-code permission names all over the app. Build a data-driven permission layer.

Current runtime contract

Use these runtime URLs:

- Base URL: <https://hn3t.pythonanywhere.com>
- OpenAPI schema: <https://hn3t.pythonanywhere.com/openapi.json>
- Interactive docs: <https://hn3t.pythonanywhere.com/docs>

Current documented API surface:

- GET /
- GET /health
- GET /api/v1/health/
- POST /api/v1/bootstrap
- POST /api/v1/auth/login
- GET /api/v1/auth/me
- POST /api/v1/auth/switch-business
- GET /api/v1/rooms/
- GET /api/v1/tasks/
- GET /api/v1/assignments/
- GET /api/v1/shifts/

Mandatory deliverables

Build these deliverables:

- 1 frontend app shell
- 2 centralized auth/session store
- 3 generated or generated-compatible API types from OpenAPI
- 4 login page
- 5 protected app layout
- 6 dashboard page
- 7 session inspector page
- 8 rooms page
- 9 tasks page
- 10 assignments page
- 11 shifts page
- 12 business switcher when memberships > 1
- 13 permission-aware navigation

14 clear empty / loading / forbidden / server-error states

Required routes

Public:

- `/login`
- `/bootstrap` (first-run only, low emphasis)

Protected:

- `/app`
- `/app/dashboard`
- `/app/session`
- `/app/rooms`
- `/app/tasks`
- `/app/assignments`
- `/app/shifts`

Future-ready route groups may be scaffolded under:

- `/app/workforce/*`
- `/app/hospitable/*`
- `/app/admin/*`
- `/app/control/*`

Required behaviors

Startup

- On app load, check for a saved token.
- If a token exists, call `/api/v1/auth/me`.
- If valid, hydrate the protected app.
- If invalid, clear session and redirect to `/login`.

Login

- Submit credentials to `POST /api/v1/auth/login`.
- Store the returned bearer token.
- Immediately call `GET /api/v1/auth/me`.
- Persist current user, memberships, roles, permissions, and active business context.
- Redirect to `/app/dashboard`.

Business switching

- If the user has multiple memberships, render a business switch control.
- On switch, call `POST /api/v1/auth/switch-business`.
- Replace the stored token with the returned token.
- Re-run `GET /api/v1/auth/me`.
- Invalidate business-scoped data caches and refetch current pages.

Data pages

- Render returned data defensively.
- Assume list shapes may stabilize later.
- Regenerate types whenever the live OpenAPI changes.

UX requirements

Build an operational admin-style UI that is:

- responsive,
- dark-theme capable,
- permission-aware,
- readable on desktop first but functional on mobile,
- designed around queue views and fast operational scanning.

Use:

- a protected shell,
- top bar,
- business indicator/switcher,
- user menu,
- collapsible navigation,
- grouped navigation sections,
- inline error panels,
- tables or cards depending on screen width.

Permission model guidance

Use backend-returned roles and permissions from `/api/v1/auth/me` for:

- nav visibility,
- route gating,
- page-level controls,

- component-level gating.

Implement helpers/selectors such as:

- `hasPermission(name)`
- `hasRole(name)`
- `canSwitchBusiness`

Do not rely on job titles for authorization.

Technology guidance

Preferred stack:

- Next.js or React + TypeScript
- typed client generated from OpenAPI where practical
- centralized session/auth store
- query caching for list endpoints
- one reusable API client with normalized error handling

Error handling rules

Handle these clearly:

- `401`: session expired, sign in again
- `403`: signed in but not authorized
- `422`: field-level validation or request issue
- `500`: readable server error state

Never hide permission errors behind generic toasts.

Future-ready architecture

Structure the frontend so it can absorb these future capabilities without major refactor:

- multi-workspace shell,
- location-scoped RBAC admin,
- profile display by location,
- Hospitable room board and inspections,
- issue / maintenance board,
- inventory and supply views,
- event timelines,
- AI widget/manifest-driven modules.

Non-negotiable constraints

- Backend authorization is the source of truth.
- UI visibility should follow backend permissions.
- Do not build a separate auth model in the frontend.
- Do not assume permission keys are final.
- Do not couple the shell only to Silver Sands or only to hospitality.
- Do not represent planned APIs as if they already exist in production.

Definition of done for the first useful build

The build is successful when:

- a user can log in,
- the session hydrates from `/api/v1/auth/me`,
- the business switcher works when applicable,
- Rooms, Tasks, Assignments, and Shifts pages render live data,
- protected routing works,
- permission and error states are readable,
- the shell is modular enough for future Workforce and Hospitable expansion.

Executive Overview

What this frontend is for

The Workforce frontend is not a single-purpose housekeeping application. It is the shared console for a multi-tenant operations platform that supports business administration, worker flows, operational modules, and future agent/integration surfaces.

The normalized product direction is:

- Workforce is the primary platform
- Hospitable / HKOps is a module or workspace inside Workforce
- One backend serves multiple scopes and workspaces
- The UI reflects authorization, business context, and location context

Platform operating model

The current planning documents consistently describe four main platform planes plus the operational workspace layer:

1. Control plane

Cross-tenant and system administration:

- business provisioning,
- feature flags,
- permission templates,
- agent registry,
- audit and export tooling.

2. Tenant plane

Business and location administration:

- users,
- role assignments,
- locations,
- reports,
- module enablement,
- business settings.

3. Worker plane

Employee-facing workflows:

- my schedule,
- time clock,
- assigned work,
- profile,
- requests.

4. Agent plane

Scoped machine and integration access:

- API credentials,
- integration runs,
- least-privilege automation,
- widget/manifest-driven tooling later.

5. Hospitable workspace

Property operations and housekeeping execution:

- rooms,
- housekeeping queues,

- inspections,
- issues,
- supply pars,
- dispatch-oriented boards.

Product boundary

Do not position Hospitable as a separate product. The current consolidated direction is to extend the existing Workforce web app into a shared console with role-aware workspaces.

That means the frontend should:

- share one shell,
- share auth/session context,
- share business and location context,
- share permission-aware navigation,
- expose different workspaces depending on user scope and enabled modules.

First real-world pilot

Silver Sands Motel is the first concrete pilot model. It validates:

- business -> location -> building -> floor -> sector -> room hierarchy,
- room readiness and housekeeping flow,
- issue handling,
- asset and supply capture,
- supervisor-oriented dashboards.

The frontend should be reusable beyond Silver Sands, but Silver Sands is the first working pilot domain that informs the initial shape of Hospitable pages.

Architecture principles the frontend must respect

Backend-first contract

The backend remains the source of truth for:

- authentication,
- business context,
- RBAC,
- operational state,
- future event/timeline history.

Scope is first-class

Users do not merely log in. They operate within:

- a selected business,
- sometimes a selected location,
- a role/permission context.

Permissions come from roles

The frontend must be built around role and permission data returned by the API. Job-title display is presentation only and must never drive access control.

Queue-first operations UX

For operational modules, prefer:

- fast scanability,
- direct status visibility,
- clear state chips,
- filterable queue views,
- low-friction drilldowns.

Avoid a CRUD-heavy back-office feel for operational pages.

Future-safe modularity

The shell must be able to host:

- Workforce core admin,
- worker flows,
- Hospitable operations,
- future maintenance / field-service / restaurant modules,
- event timelines,
- AI-generated or manifest-defined widgets.

Recommended immediate frontend outcome

The first frontend should not try to solve the entire roadmap. It should deliver a solid foundation:

- 1 authenticated app shell,
- 2 session hydration from live API,
- 3 business switching,
- 4 permission-aware navigation,
- 5 connected live-data pages for rooms, tasks, assignments, and shifts,
- 6 dashboard/session pages that expose user context clearly,
- 7 structural readiness for future workspaces and operational boards.

Current and Planned API Reference

This document separates:

- Current documented API surface — endpoints that the current Manus brief treats as available now.
- Planned target API surface — normalized architecture targets from the master planning documents.

Do not confuse the two. The current live OpenAPI is the runtime source of truth.

A. Runtime base

- Base URL: <https://hn3t.pythonanywhere.com>
 - OpenAPI schema: <https://hn3t.pythonanywhere.com/openapi.json>
 - Interactive docs: <https://hn3t.pythonanywhere.com/docs>
-

B. Current documented API surface

Public / bootstrap / health

Method	Path	Purpose	Notes
GET	/	root or service check	basic reachability
GET	/health	health check	simple service validation
GET	/api/v1/health/	versioned health check	preferred API health path
POST	/api/v1/bootstrap	first-run system bootstrap	should return 403 after initial users exist

Authentication and business context

Method	Path	Purpose	Notes
POST	/api/v1/auth/login	authenticate user	returns bearer token and business context
GET	/api/v1/auth/me	hydrate current session	returns user, memberships, roles, permissions, business context
POST	/api/v1/auth/switch-business	switch active business context	replaces token and changes business-scoped context

Current authenticated data pages

Method	Path	Purpose	Notes
GET	/api/v1/rooms/	list rooms	current schema may still be generic; render defensively
GET	/api/v1/tasks/	list tasks	current schema may still be generic; render defensively
GET	/api/v1/assignments/	list assignments	business-scoped/auth-scoped
GET	/api/v1/shifts/	list shifts	business-scoped/auth-scoped

C. Current auth contract details

POST /api/v1/auth/login

Request body

```
{
  "email": "user@example.com",
  "password": "secret",
  "business_id": null
}
```

Example response

```
{
  "access_token": "...",
  "token_type": "bearer",
  "business_id": "uuid",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "is_active": true
  }
}
```

Session hydration sequence

After login:

- 1 save `access_token`
- 2 call GET `/api/v1/auth/me`
- 3 store:
 - current user
 - active business id
 - memberships
 - roles
 - permissions

- 1 enter the protected app

Business switch sequence

When membership count > 1:

- 1 call `POST /api/v1/auth/switch-business`
 - 2 replace stored token
 - 3 re-run `GET /api/v1/auth/me`
 - 4 invalidate business-scoped caches
 - 5 refetch list pages
-

D. Current error handling expectations

Status	Meaning	Frontend requirement
401	unauthenticated / expired session	clear token and route to login
403	signed in but not authorized	show readable in-page denial
422	validation or request issue	show request/field feedback
500	backend error	show readable server error state

Important behavior notes

- Protected endpoints require `Authorization: Bearer <access_token>`.
 - `POST /api/v1/bootstrap` is first-run only.
 - Once initial users exist, bootstrap should return `403`.
 - Permission failures must be visible and explicit.
-

E. Current UI-facing contract expectations

The current frontend should assume:

- `/api/v1/auth/me` is the session authority,
- the app should be able to render user roles and permissions,
- rooms/tasks/assignments/shifts are list-oriented views,
- some response schemas may still be broad or evolving.

Use generated types where possible, but keep rendering resilient enough to survive early backend evolution.

F. Planned target API architecture

The master planning documents describe a larger target API shape centered on explicit slices:

- `/api/v1/auth/*`
- `/api/v1/control/*`
- `/api/v1/tenant/*`
- `/api/v1/worker/*`
- `/api/v1/agent/*`

This is the planned architectural grouping, not a guarantee that all endpoints already exist in the live runtime.

G. Planned RBAC and profile endpoints

These endpoints are described in the normalized planning material as target structures the frontend should be ready to consume later.

Location-scoped RBAC roles

Method	Path	Planned purpose
GET	<code>/api/rbac/locations/{location_id}/roles</code>	list location roles
POST	<code>/api/rbac/locations/{location_id}/roles</code>	create location role
PATCH	<code>/api/rbac/locations/{location_id}/roles/{role_id}</code>	update location role
DELETE	<code>/api/rbac/locations/{location_id}/roles/{role_id}</code>	delete location role

Location-scoped assignments

Method	Path	Planned purpose
GET	<code>/api/rbac/locations/{location_id}/assignments</code>	list role assignments for location
POST	<code>/api/rbac/locations/{location_id}/assignments</code>	create assignment
DELETE	<code>/api/rbac/locations/{location_id}/assignments/{assignment_id}</code>	delete assignment

User profile in location context

Method	Path	Planned purpose
GET	/api/locations/{location_id}/users/{user_id}/profile	get display profile and role-derived title for location

H. Planned module/API groups the frontend should anticipate

These groups are strongly implied by the product direction but should be treated as future targets until the backend exposes them.

Workforce core / admin targets

Potential capabilities:

- businesses
- locations
- memberships
- users
- roles
- permissions
- reports
- audit logs
- settings

Hospitable / HKOps targets

Potential capabilities:

- room board and room details
- housekeeping task queues
- inspections
- issues / maintenance flags
- room assets
- supply pars / inventory signals
- bulk room actions
- dashboard and dispatch summaries

Event / timeline targets

Planned Phase 1 direction:

- append-only Postgres-backed events,
- timeline-style API access,

- later projection/publishing infrastructure.

Agent / AI widget targets

Planned future direction:

- manifest-driven widget definitions,
- permission validation,
- preview/publish workflow,
- scoped agent access.

I. Proposed future contract categories for frontend organization

Even before those APIs exist, organize the frontend API layer in categories such as:

- `auth`
- `health`
- `rooms`
- `tasks`
- `assignments`
- `shifts`
- `rbac`
- `businesses`
- `locations`
- `profiles`
- `inspections`
- `issues`
- `inventory`
- `events`
- `dashboard`
- `agents`
- `widgets`

This keeps the client modular and prevents a later rewrite.

J. Non-negotiable distinction

Current, use now

Use the live OpenAPI / current documented endpoints for working pages:

- login
- session hydration
- business switching
- rooms
- tasks
- assignments
- shifts

Planned, design for later

Design the shell and API layer so it can absorb:

- full multi-workspace architecture,
- location-aware RBAC admin,
- profile-by-location,
- Hospitable operational boards,
- event timeline surfaces,
- AI widget tooling.

Do not present planned endpoints as currently live unless verified against the live OpenAPI.

Frontend Product Architecture

Frontend mission

The frontend should act as a shared Workforce console with a reusable app shell, not as a bundle of unrelated screens.

The design target is:

- secure,
 - modular,
 - context-aware,
 - permission-aware,
 - easy to extend with future workspaces.
-

1. Recommended route map

Public routes

- /login
- /bootstrap

Protected shell

- /app

Protected current pages

- /app/dashboard
- /app/session
- /app/rooms
- /app/tasks
- /app/assignments
- /app/shifts

Future-ready route groups

- /app/workforce/*
- /app/hospitable/*
- /app/admin/*
- /app/control/*

2. Shell structure

The shell should include persistent app-level controls:

Top bar

- app/workspace title
- active business indicator
- optional active location indicator later
- environment badge
- user menu
- logout action

Sidebar

Recommended grouping:

Overview

- Dashboard

- Session

Operations

- Rooms
- Tasks
- Assignments
- Shifts

Future groups

- People
- Inventory
- Scheduling
- Reports
- Admin / Setup
- Hospitable

Shell rules

- collapse/expand on desktop
- drawer on mobile
- remember nav state with local storage
- highlight active section
- hide or soften items the user cannot access
- keep the shell reusable across current and future workspaces

3. Business and location context

The system is multi-tenant and scope-aware.

Business context

The user may belong to multiple businesses. The frontend must:

- show the current business,
- let the user switch if multiple memberships exist,
- rehydrate the session after switching,
- clear or invalidate business-scoped cached data.

Location context

Location is central to the long-term platform. Even if the first frontend does not yet implement a full location selector, the architecture should be ready for:

- location-aware workspaces,
 - location-filtered permissions,
 - location-scoped role assignment UI,
 - location-specific profile display.
-

4. Page responsibilities

Dashboard

Purpose:

- landing page after login,
- expose session context clearly,
- provide navigation into working modules,
- help validate that the auth/session model is functioning.

Suggested dashboard sections:

- current user summary
- active business summary
- role chips
- permission list or counts
- memberships overview
- health/API summary
- quick links to working modules

Session page

Purpose:

- transparent debugging/admin view of current auth state.

Show:

- user id
- email
- active status
- active business id
- memberships
- roles

- permissions
- token present/absent state

Rooms page

Purpose:

- current live operational room list.

UX pattern:

- table on desktop
- cards on narrow screens
- explicit empty state
- explicit forbidden state
- refresh action
- future room-board expansion path

Tasks page

Purpose:

- current live task list.

UX pattern:

- queue-first layout
- table/card hybrid
- future filters scaffold
- future grouping by status/assignee/location

Assignments page

Purpose:

- current list view for assignments.

UX pattern:

- readable list/table
- permission denied state
- empty state
- future assignment actions scaffold

Shifts page

Purpose:

- current list view for shifts.

UX pattern:

- readable list/table
 - empty state
 - future schedule and staffing drill-in path
-

5. Permission-aware navigation and view logic

Use permission data returned from the session API.

Recommended behavior

- If a route is known to require auth, wrap it in `<RequireAuth>`.
- If a route or action is permission-sensitive, gate it via helper/selectors.
- If permission mapping is still evolving, err on:
 - allowing navigation,
 - handling 403 cleanly,
 - not hard-coding policy everywhere.

Recommended helpers

- `hasPermission(name)`
 - `hasRole(name)`
 - `canSwitchBusiness`
 - `isOwner`
 - `isAuthenticated`
-

6. UI style direction

The design system should feel like an operational console:

- high contrast,
- fast scanning,
- compact but readable,
- status-aware,
- desktop-first without ignoring mobile.

Avoid:

- over-styled marketing-site aesthetics,
- hidden state,

- excessive modal dependence,
- overly dense enterprise clutter.

Prefer:

- cards for summaries,
 - tables for lists,
 - chips/badges for state,
 - inline alerts for errors,
 - drawers/panels for future operational detail.
-

7. Operational UX principles from the product docs

The frontend should follow these patterns as future modules are added:

Queue-first over CRUD-first

Operational teams need to see:

- what needs attention,
- what is blocked,
- what changed,
- what is late,
- what is ready.

Explicit state over vague booleans

Rooms, tasks, inspections, issues, shifts, and approvals should have readable lifecycle states.

Fast field capture

The UI should support work performed in the field, not only at a desk.

Permission-aware workspaces

Navigation, widgets, and views should reflect who the user is and where they are operating.

8. Future workspace structure

The long-term shell should support workspace switching or workspace grouping such as:

- Superadmin / Control
- Tenant Admin
- Workforce
- Hospitable

A single user may only see one or two of these based on permissions. Do not assume every user sees the whole product.

9. Frontend outcome to target now

For the first real build, the frontend should prove:

- the shell works,
 - auth/session hydration works,
 - business context switching works,
 - current live list pages work,
 - navigation and permissions are handled cleanly,
 - the architecture is ready for the next slice of Workforce and Hospitable features.
-

UI System and Design Tokens

Design goal

Build a UI system that is operational, readable, and expandable. The first frontend should already contain the core shell and design primitives that later pages can reuse.

1. Theme model

The planning docs call for a tokenized dark-theme-capable UI. Use CSS variables or equivalent design tokens.

Minimum tokens

```
:root {
  --bg: #0f1115;
  --bg-elevated: #171a21;
  --text: #f3f5f7;
  --text-muted: #a7b0ba;
  --border: #2a3240;
  --primary: #4f8cff;
  --primary-contrast: #ffffff;
  --success: #20b26b;
  --warning: #e3a53f;
  --danger: #d65b5b;
  --nav-bg: #121722;
  --nav-text: #d7dce3;
  --nav-active: #23314d;
}
```

These values are a starter set. Manus may refine the palette, but the token structure should remain.

Theme behavior

- support dark by default or dark-ready theming,

- persist theme state in local storage,
 - optionally support a user preference field later.
-

2. Layout system

Desktop

- left sidebar
- top bar
- main content pane
- page header + content region
- cards/tables within responsive content width

Mobile

- top bar
 - hamburger/drawer nav
 - stacked cards
 - simpler density and fewer columns
-

3. Core reusable components

Create reusable primitives such as:

- AppShell
- TopBar
- Sidebar
- PageHeader
- StatCard
- DataTable
- EmptyState
- ErrorState
- ForbiddenState
- LoadingState
- StatusBadge
- BusinessSwitcher
- UserMenu

- **PermissionGate**

These should be generic enough for future Workforce and Hospitable modules.

4. State and badge styling

Operational products need strong state visibility. Standardize status display through chips/badges.

Use badges for

- active/inactive
- success/failure
- blocked
- warning
- permission scope
- role markers
- room/task/issue states later

Principles

- color alone should not carry meaning,
 - labels should be explicit,
 - the same state should look the same across pages.
-

5. List and detail patterns

Current list pages

Use:

- table on desktop,
- stacked cards on smaller screens,
- refresh control,
- empty state,
- forbidden state,
- inline error state.

Future detail patterns

Prepare to support:

- drill-down side panels,
- row details,

- lightweight drawers,
- timeline/event sections,
- inline action menus.

Avoid designing only around full-page form navigation.

6. Navigation design rules

Grouping

Use grouped sections rather than a flat long menu:

- Overview
- Operations
- People
- Inventory
- Scheduling
- Reports
- Admin / Setup
- Hospitable (when enabled)

Behavior

- highlight active section,
 - support collapsed nav on desktop,
 - store collapsed state,
 - hide or soften inaccessible items,
 - do not show raw internal IDs in the UI.
-

7. Form and error patterns

Forms should:

- show inline labels,
- show validation near fields,
- show submit/loading state,
- provide readable server error summaries.

Error handling should include dedicated UI for:

- unauthenticated,

- forbidden,
 - validation error,
 - server error,
 - empty dataset.
-

8. Dashboard composition

The first dashboard should be simple and useful.

Recommended blocks:

- current user card
- active business card
- memberships block
- role chips block
- permissions block or count summary
- quick links block
- API health/status block

This makes the dashboard both functional and diagnostic.

9. Future Hospitable UI direction

As Hospitable pages come online, prefer:

- room-board style over raw table-only design,
- state chips for room/task/inspection/issue lifecycles,
- sector/group navigation,
- bulk actions,
- fast assignment and status changes,
- mobile-friendly task execution views.

The initial shell should not prevent this future direction.

Technical Implementation Guide

1. Recommended stack

Manus can choose the exact implementation, but the target stack should be close to:

- Framework: Next.js or React

- Language: TypeScript
 - API typing: generated or generated-compatible types from OpenAPI
 - State: centralized auth/session store
 - Data fetching: query-cached list retrieval
 - Routing: protected routes / route guards
 - Styling: tokenized theme system
 - Error handling: normalized API error wrapper
-

2. Suggested project structure

```
src/  
  app/ or pages/  
    login/  
    bootstrap/  
    app/  
      dashboard/  
      session/  
      rooms/  
      tasks/  
      assignments/  
      shifts/  
  components/  
    shell/  
    common/  
    states/  
    tables/  
    forms/  
  features/  
    auth/  
    session/  
    businesses/  
    rooms/  
    tasks/  
    assignments/  
    shifts/  
  lib/  
    api/  
    auth/  
    permissions/  
    utils/  
  store/  
  styles/  
  types/
```

Keep features modular rather than placing all API or UI logic into one folder.

3. Session state model

Use a centralized store for:

```
type SessionState = {  
  accessToken: string | null  
  currentUser: {
```

```
  id: string
  email: string
  is_active: boolean
} | null
activeBusinessId: string | null
memberships: unknown[]
roles: string[]
permissions: string[]
isAuthenticated: boolean
isBootstrapped?: boolean
}
```

Helpful derived selectors

- `canSwitchBusiness`
 - `hasPermission(name)`
 - `hasRole(name)`
 - `isOwner`
 - `isAuthenticated`
-

4. API client requirements

Build one API layer that:

- stores base URL centrally,
- attaches bearer tokens automatically,
- parses JSON safely,
- normalizes error shape,
- supports token replacement after business switching,
- is modular enough for future endpoint groups.

Suggested normalized error shape

```
type ApiError = {
  status: number
  message: string
  details?: unknown
}
```

Required client behaviors

- on `401`, allow auth layer to clear session,
 - on `403`, preserve the current route and show denial where possible,
 - on `422`, expose request/field details,
 - on `500`, expose a readable server error state.
-

5. OpenAPI and type generation strategy

Treat the live OpenAPI as the source of truth for currently available endpoints.

Recommended workflow:

- 1 fetch/generate types from `https://hn3t.pythonanywhere.com/openapi.json`
- 2 commit generated client artifacts or generation script
- 3 regenerate whenever the backend changes
- 4 keep handwritten wrappers thin and isolated

Important

Because the backend is still evolving, the UI should render current responses defensively even when using generated types.

6. Data fetching strategy

For current list pages:

- fetch on page load,
- cache results,
- support explicit refresh,
- invalidate/refetch on business switch.

Do not build optimistic mutations for the initial version unless live write endpoints are actually present and stable.

7. Auth and routing flow

Startup

- read token from storage
- if present, call `/api/v1/auth/me`
- if success, hydrate session
- if fail, clear token and go to `/login`

Login

- submit to `/api/v1/auth/login`
- save token
- call `/api/v1/auth/me`
- enter protected area

Logout

- clear token
- clear session state
- clear cached queries
- go to `/login`

Business switch

- call `/api/v1/auth/switch-business`
 - replace token
 - rehydrate `/api/v1/auth/me`
 - invalidate current business data
-

8. Defensive rendering strategy

The current backend is early enough that some payload shapes may still evolve.

For the first build:

- avoid brittle assumptions,
 - favor resilient list renderers,
 - show raw key/value fallback carefully in development if needed,
 - regenerate types as the API stabilizes.
-

9. Security and environment handling

Use environment configuration for:

- API base URL
- environment badge value
- optional feature flags

Do not:

- expose raw tokens visibly in production UI,
 - duplicate authorization logic in ad hoc ways,
 - store secrets in source.
-

10. Future extension strategy

Organize the app so future modules can be added without rewriting the shell:

future API domains

- rbac
- businesses
- locations
- profiles
- inspections
- issues
- inventory
- events
- dashboard
- widgets

future UI surfaces

- location selector
- role/assignment admin
- room board
- inspection board
- issue board
- event timeline
- inventory views
- AI widget surfaces

This is why the shell, store, and API layers must be modular from the start.

Delivery Plan and Acceptance

Goal of the first delivery slice

Get an operational base product working with:

- a browser-visible frontend shell,
 - authenticated session flow,
 - live API connectivity,
 - connected pages for dashboard, rooms, tasks, assignments, and shifts,
 - enough modularity for the next phase.
-

1. Recommended build sequence

Phase 1 — runtime contract and shell

- verify API base and OpenAPI
- generate typed client
- build API wrapper
- build auth/session store
- implement protected routing
- implement top-level shell

Phase 2 — auth flow

- login page
- bootstrap page
- session hydration from `/api/v1/auth/me`
- logout flow
- business switching

Phase 3 — working pages

- dashboard
- session inspector
- rooms page
- tasks page
- assignments page
- shifts page

Phase 4 — hardening

- empty/loading/forbidden/error states
- defensive rendering cleanup
- mobile responsiveness
- navigation cleanup
- permission-driven visibility rules

Phase 5 — handoff readiness

- route map documented
- API generation workflow documented
- known gaps documented
- next-phase extension points documented

2. Minimum acceptance criteria

The first frontend is acceptable when all of the following are true:

Auth/session

- user can log in with email/password
- bearer token is stored and used correctly
- `/api/v1/auth/me` hydrates the session
- logout clears session properly
- expired token routes back to login cleanly

Business context

- business switcher is shown when multiple memberships exist
- switching business replaces the token
- session is rehydrated after switching
- business-scoped data refreshes after switching

Current live data pages

- rooms page renders data or empty state
- tasks page renders data or empty state
- assignments page renders data or empty state
- shifts page renders data or empty state

Error handling

- `401` is readable and routes correctly
- `403` is readable and not silently swallowed
- `422` shows meaningful feedback
- `500` shows a readable server error state

Architecture quality

- the shell is modular
- permissions are handled centrally
- API access is centralized
- current and future endpoint groups are not tangled together

3. Recommended testing checklist

Smoke tests

- service base URL reachable
- OpenAPI fetch/generation works
- login request succeeds with valid credentials
- invalid login shows error
- `/auth/me` failure clears session

Navigation tests

- protected routes redirect when logged out
- sidebar renders current pages
- active nav state is visible
- business switcher appears only when appropriate

Data-page tests

- rooms success / empty / forbidden / server-error
- tasks success / empty / forbidden / server-error
- assignments success / empty / forbidden / server-error
- shifts success / empty / forbidden / server-error

Session tests

- session page shows user context
- roles and permissions render
- token visibility is safe in production mode

UI tests

- layout works on narrow screens
- nav collapse persists
- dark-theme tokens are consistent
- loading states are not jarring

4. Coordination expectations

Manus owns

- browser-visible frontend shell

- typed-client generation workflow
- connected UI construction
- frontend modularity and design execution

Copilot / backend work owns

- backend fixes
- API mismatches
- migrations
- seed data
- contract stabilization

User / operator owns

- deployment targets
- environment variables
- runtime approvals
- browser verification

ChatGPT-type planning work owns

- specs
- acceptance criteria
- handoff documents
- debugging guidance
- scope normalization

5. What should happen immediately after first delivery

Once the first slice works, the next frontend phase should likely move into one of these:

- 1 stronger room/task operational UX,
- 2 location-aware navigation and context,
- 3 RBAC admin surfaces,
- 4 Hospitable dashboard / room board / inspections,
- 5 event and audit timeline surfaces.

The first build should make those possible without major shell rewrites.

Assumptions and Guardrails

1. What is assumed to be true

The package assumes the following based on the current planning corpus:

- Workforce is the umbrella platform.
 - Hospitable / HKOps is a workspace/module within Workforce.
 - Multi-business tenancy and role-based access are core requirements.
 - Location-scoped RBAC is the normalized long-term access model.
 - The current frontend should use the live API for real working pages.
 - The longer-term frontend should support multiple workspaces and operational modules.
-

2. What is current versus planned

Current

Documented as available now:

- login
- session hydration
- business switching
- rooms
- tasks
- assignments
- shifts
- health endpoints
- bootstrap route

Planned

Described in architecture docs but not guaranteed live right now:

- full control/tenant/worker/agent API group expansion
- location RBAC management UI
- profile-by-location UI
- inspections/issues/inventory/event timelines
- advanced room board and dispatch UX
- AI widget/manifest surfaces

Do not blur this line.

3. Frontend constraints

Do not

- build a disconnected hospitality-only frontend
- use job titles as authorization data
- hard-code permission policy throughout many files
- assume payload schemas are fully stable
- pretend proposed APIs already exist
- over-couple the app shell to one pilot property

Do

- centralize API access
 - centralize session state
 - make permissions data-driven
 - keep routes modular
 - keep list rendering defensive
 - use the live OpenAPI as current contract authority
-

4. Backend/contract uncertainty handling

If Manus discovers that the live OpenAPI differs from older planning notes:

- 1 trust the live OpenAPI for runtime implementation,
 - 2 keep the shell future-ready,
 - 3 document mismatches clearly,
 - 4 avoid building fake live behavior.
-

5. UX guardrails

The first version should be:

- readable,
- operational,
- diagnostic where useful,
- easy to debug,
- easy to extend.

It should not try to become a fully polished enterprise suite before the current runtime loop is working.

6. Future-proofing note

The planning documents strongly indicate future expansion into:

- richer HKOps operational boards,
- event timeline APIs,
- AI widget generation,
- additional service domains beyond hospitality.

That means the frontend should be a stable shell and contract-driven client, not a narrow one-off prototype.
